

CO2 - AT2 - Part I

Industry Problem Task - Dynamic Programming & Monte Carlo Methods

Industry Problem:

**Cloud Infrastructure Performance Optimization using
Dynamic Programming & Monte Carlo Methods**

N.PARNITA

192525322

```
# =====
# CO2 - AT1 PART 1
# Dynamic Programming & Monte Carlo Methods
# =====

# Import Required Libraries

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.mixture import GaussianMixture

from scipy.stats import ttest_ind

import random

# Display Settings

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

# Random Seed

np.random.seed(42)
random.seed(42)

print("="*60)
print("All Libraries Imported Successfully")
print("="*60)

=====
All Libraries Imported Successfully
=====
```

```

# =====
# IMPORT LIBRARIES
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from scipy.stats import ttest_ind

np.random.seed(42)

# 6 Features
features = [
    "Reliability",
    "Consistency",
    "Scalability",
    "Flexibility",
    "Adaptability",
    "Redundancy"
]

# Generate Dataset using GMM

means = [
    [90,88,86,84,87,20],
    [75,73,72,70,71,35],
    [60,58,56,57,59,50]
]

cov = np.diag([15,15,15,15,15,10])

data=[]

for m in means:
    temp=np.random.multivariate_normal(m,cov,40)
    data.append(temp)

data=np.vstack(data)

gmm=GaussianMixture(n_components=3,random_state=42)
gmm.fit(data)

```

```
samples,_=gmm.sample(100)

df=pd.DataFrame(samples,columns=features)

df=df.clip(0,100).round(2)

print("Dataset Shape :",df.shape)
print("\nDataset Created Successfully\n")
```

Dataset Shape : (100, 6)

Dataset Created Successfully

```
# =====
# TABLE 1 & TABLE 2
# =====

print("TABLE 1 : DATASET SUMMARY")
display(df.describe().round(2))

print("\nTABLE 2 : RANDOM 10 RECORDS")

sample=df.sample(10,random_state=42)

display(sample)
```

TABLE 1 : DATASET SUMMARY

	Reliability	Consistency	Scalability	Flexibility	Adaptability	Redundancy
count	100.00	100.00	100.00	100.00	100.00	100.00
mean	75.01	74.37	71.19	70.27	72.90	35.57
std	13.46	12.45	13.16	10.72	12.07	12.29
min	50.34	54.34	46.50	53.02	51.26	15.89
25%	62.45	61.37	58.07	59.04	61.99	22.74
50%	73.88	74.88	72.72	70.28	69.82	36.12
75%	88.58	86.40	82.53	79.83	85.97	48.09
max	96.64	95.58	92.57	89.53	93.88	55.98

TABLE 2 : RANDOM 10 RECORDS

	Reliability	Consistency	Scalability	Flexibility	Adaptability	Redundancy
83	90.24	85.95	80.83	81.52	87.79	19.97
53	67.30	58.33	57.61	55.61	61.35	49.68
70	94.03	84.88	80.60	78.74	88.05	28.07
45	59.74	56.73	54.51	58.39	60.55	49.15
44	59.99	60.75	50.16	61.49	66.29	50.22
39	61.09	62.52	58.23	53.82	60.41	48.04
22	71.96	71.77	68.56	70.22	62.87	34.16
80	96.05	88.26	87.38	87.77	83.14	20.72
10	75.34	70.13	74.41	70.58	74.24	37.81
0	67.81	70.47	73.31	67.16	70.66	31.64

```

# =====
# DYNAMIC PROGRAMMING
# MONTE CARLO
# TABLE 3
# =====

dp={}
mc={}

for feature in features:

    values=df[feature].values

    # Dynamic Programming
    best=values[0]

    for i in range(1,len(values)):
        best=max(best,values[i])

    dp[feature]=best

    # Monte Carlo
    simulation=[]

    for i in range(1000):

        sample=np.random.choice(values,20,replace=True)

        simulation.append(np.mean(sample))

    mc[feature]=np.mean(simulation)

comparison=pd.DataFrame({

    "Feature":features,

    "Dynamic Programming":list(dp.values()),

    "Monte Carlo":np.round(list(mc.values()),2)

})

```

```
print("TABLE 3 : DP vs MONTE CARLO")
```

```
display(comparison)
```

TABLE 3 : DP vs MONTE CARLO

	Feature	Dynamic Programming	Monte Carlo
0	Reliability	96.64	75.02
1	Consistency	95.58	74.44
2	Scalability	92.57	71.13
3	Flexibility	89.53	70.29
4	Adaptability	93.88	72.91
5	Redundancy	55.98	35.59

```

# =====
# TABLE 4
# STATISTICS
# GRAPHS
# =====

t,p=ttest_ind(list(dp.values()),list(mc.values()))

stats_table=pd.DataFrame({

    "T-Statistic":[round(t,3)],

    "P-Value":[round(p,5)]

})

print("TABLE 4 : STATISTICAL EVALUATION")

display(stats_table)

# GRAPH 1

plt.figure(figsize=(10,5))

x=np.arange(len(features))

plt.bar(x-0.2,list(dp.values()),0.4,label="Dynamic Programming")

plt.bar(x+0.2,list(mc.values()),0.4,label="Monte Carlo")

plt.xticks(x,features)

plt.xlabel("Features")

plt.ylabel("Performance Score")

plt.title("Feature Performance Comparison")

plt.legend()

plt.show()

```



```

# GRAPH 2

plt.figure(figsize=(10,5))

plt.plot(features,list(dp.values()),marker='o',label="Dynamic Programming")

plt.plot(features,list(mc.values()),marker='s',label="Monte Carlo")

plt.xlabel("Features")

plt.ylabel("Performance Score")

plt.title("Dynamic Programming vs Monte Carlo")

plt.legend()

plt.grid(True)

plt.show()

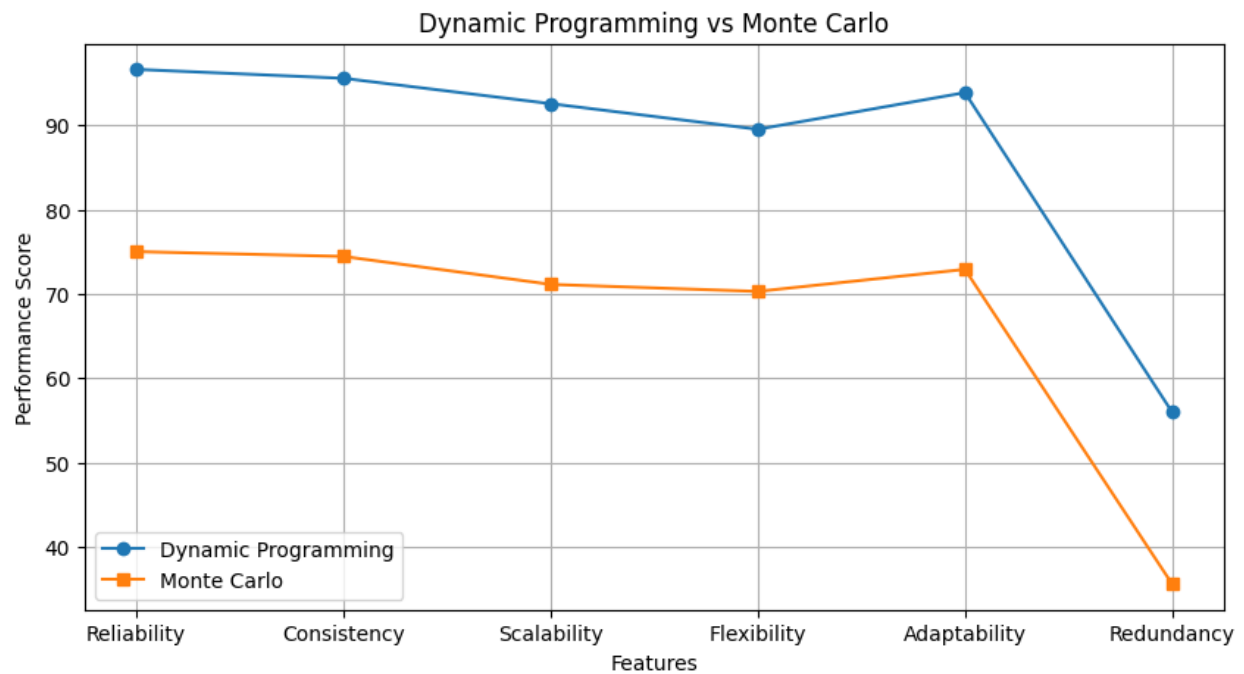
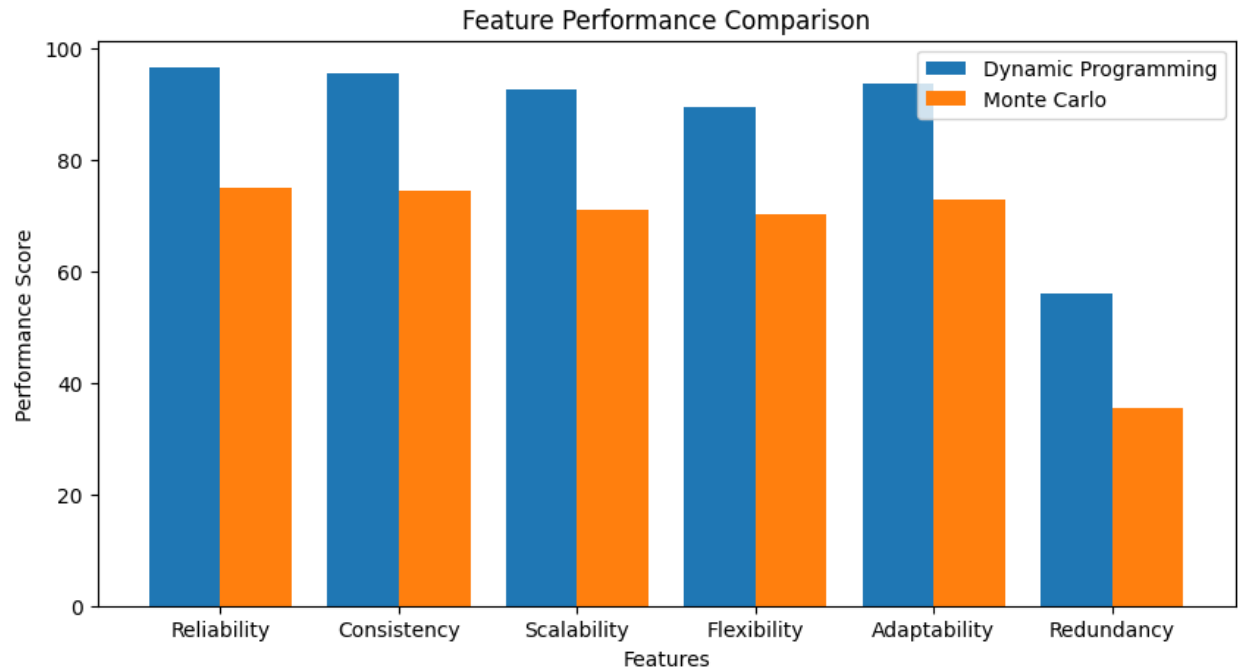
print()

if p<0.05:
    print("Result : Statistically Significant (p < 0.05)")
else:
    print("Result : Not Statistically Significant (p >= 0.05)")

```

TABLE 4 : STATISTICAL EVALUATION

	T-Statistic	P-Value
0	2.335	0.04171



Result : Statistically Significant ($p < 0.05$)